

Índice

Cazador de Recompensas – (🐾 decorator-profugos)	1
¿Por qué nos dan la interfaz IProfugo desde el comienzo?	1
Construcción técnica del Decorator	1
Cómo resuelve cada entrenamiento su responsabilidad	2
Diagrama de clases decorator	3
Conclusión: la gracia del patrón	4

Cazador de Recompensas – (🐾 decorator-profugos)

¿Por qué nos dan la interfaz IProfugo desde el comienzo?

Es un indicio fuerte de que vamos a usar Decorator. Sin interfaz, el patrón no es posible, necesitamos un tipo común (IProfugo) para que tanto el componente concreto (Profugo) como los decoradores sean intercambiables. Si el enunciado fuese distinto (ej: solo un Profugo concreto sin evolución), bastaba una clase sola y no hacía falta la interfaz. Al darla desde el principio, nos están diciendo «vas a necesitar polimorfismo sobre IProfugo», que es exactamente lo que pide el Decorator cuando los prófugos evolucionan apilando entrenamientos.

Construcción técnica del Decorator

Tenemos cuatro actores:

Componente	Rol
IProfugo	Contrato. Define todas las operaciones: <code>getInocencia()</code> , <code>getHabilidad()</code> , <code>esNervioso()</code> , <code>volverseNervioso()</code> , <code>dejarDeEstarNervioso()</code> , <code>reducirHabilidad()</code> , <code>disminuirInocencia()</code>
Profugo	Componente concreto. Implementa IProfugo con estado real. Es el objeto base sin entrenamiento.
ProfugoDecorator (abstracta)	Implementa IProfugo y tiene una referencia a un IProfugo envuelto (el «wrappee»). Por defecto, delega todos los métodos al wrappee.

Componente	Rol
Concretos (ArtesMarciales, EntrenamientoElite, ProteccionLegal)	Extienden ProfugoDecorator y overridean solo los métodos que modifican.

ProfugoDecorator es la clave: no repite lógica, solo pasa el mensaje al wrappee. En cada override de los concretos hacemos algo antes o después de llamar a `super.metodo()`, o evitamos la llamada si queremos bloquear el comportamiento.

El armado se hace por composición en el constructor:

```
1 IProfugo p = new ProteccionLegal(
2     new ArtesMarciales(
3     new Profugo(...)));
```

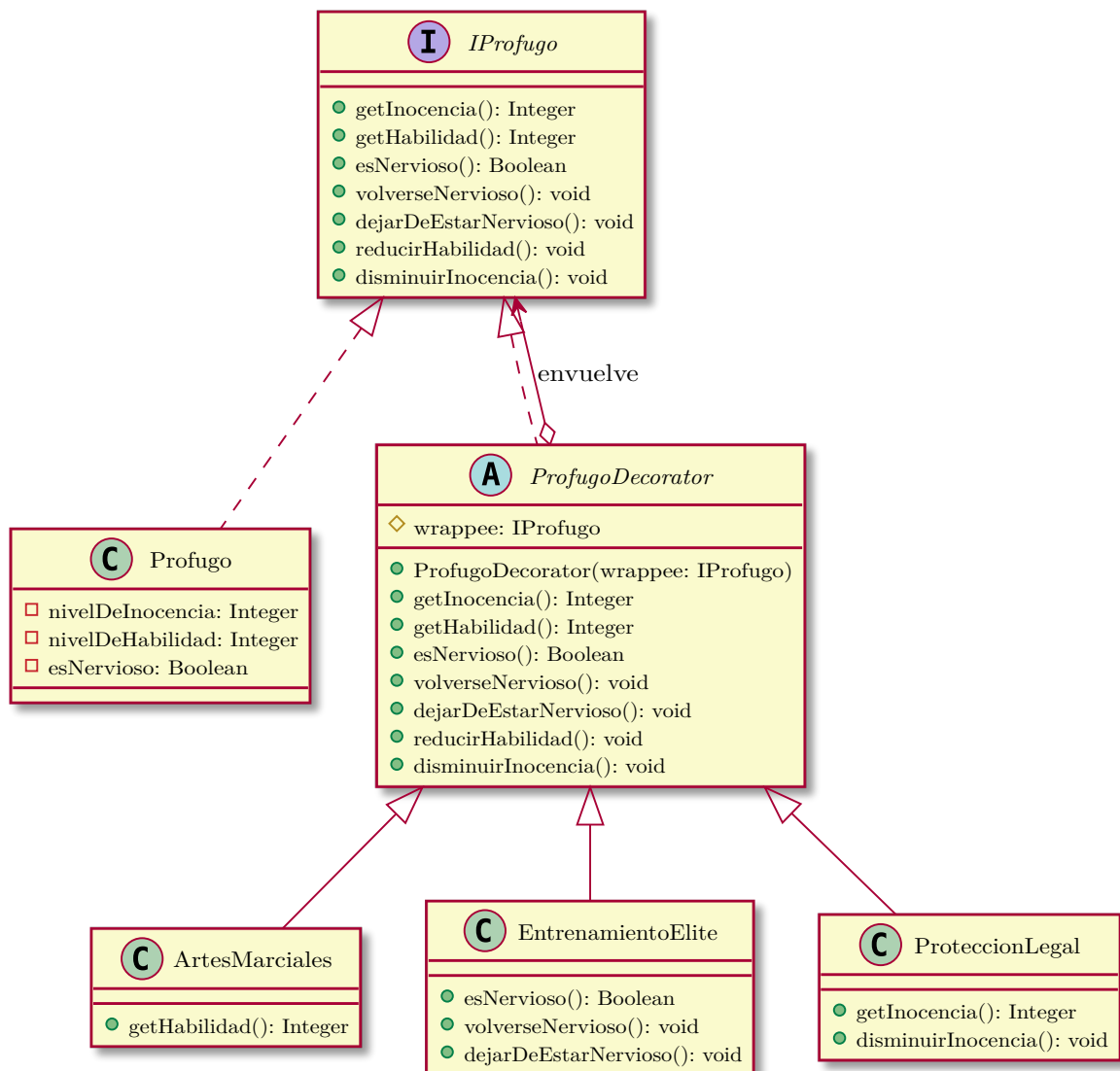
El orden importa: el decorador más externo envuelve al siguiente, que envuelve al siguiente, etc. `p` es un `IProfugo` y el código cliente no sabe ni le importa cuántas capas tiene.

Cómo resuelve cada entrenamiento su responsabilidad

Decorador	Método overrideado	Comportamiento
ArtesMarciales	<code>getHabilidad()</code>	Obtiene el valor del wrappee (<code>super.getHabilidad()</code>), lo duplica, lo capsula a 100 y lo devuelve. El resto de métodos se delegan sin cambios.
EntrenamientoElite	<code>esNervioso()</code>	Siempre devuelve <code>false</code> .
EntrenamientoElite	<code>volverseNervioso()</code>	No hace nada (bloquea el efecto).
EntrenamientoElite	<code>dejarDeEstarNervioso()</code>	No hace nada (ya es falso siempre).
ProteccionLegal	<code>getInocencia()</code>	Devuelve <code>Math.max(40, super.getInocencia())</code> .

Decorator	Método overrideado	Comportamiento
ProteccionLegal	<code>disminuirInocencia()</code>	Puede delegar al wrappee pero la lectura queda pisada por el getter. O bien, directamente implementa el piso en 40.

Diagrama de clases decorator



Leyenda de flechas: $\cdots \rightarrow$ — implements (realización de interfaz), $\rightarrow$ — extends (herencia de clase), $\diamond \rightarrow$ — agregación (envuelve, tiene-un).

ProfugoDecorator tiene solo dos cosas que la diferencian de IProfugo:

1. **El atributo wrappee: IProfugo** — la referencia al objeto envuelto.

2. **Delegación pura** — implementa todos los métodos de **IProfugo** llamando al mismo método del **wrappee**, sin agregar lógica.

Los decoradores concretos extienden **ProfugoDecorator** y **overlinean solo los métodos que modifican**, llamando a **super.metodo()** (que delega al **wrappee**) y alterando el resultado. Sin el **wrappee** no hay decoración posible. Es el núcleo del patrón.

ProfugoDecorator implementa **todos** los métodos de **IProfugo** porque es un **wrapper completo**. Desde afuera, un **ProfugoDecorator** se comporta exactamente como un **IProfugo** — el cliente (cualquier código que use un **IProfugo**) no sabe si está hablando con un **Profugo** base, un decorador, o una pila de decoradores.

Conclusión: la gracia del patrón

Cada decorador agrega una responsabilidad única sin modificar la clase base, y se pueden combinar arbitrariamente sin explosión de subclases. La interfaz **IProfugo** es el pegamento que hace esto posible: sin ella, no podríamos apilar comportamientos de forma transparente.